Universidad El Bosque
Programa: Ingeniería de Sistemas
Asignatura: Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte

Integrantes:

- Gabriella Castro
- Lina Patiño
- Julian Samuel Rodriguez Hernandez

Bogotá D.C., noviembre 12 de 2026

PROPUESTA DE PROYECTO FINAL
BountyLine: red social de recompensas en un mundo futurista

Tabla de contenido

1. Contexto del proyecto 2
2. Problemática 2
3. Objetivo general 2
4. Objetivos específicos 2
5. Descripción de la solución propuesta 3
6. Justificación del modelo de datos 4
7. Entidades principales identificadas (modelo preliminar) 5
8. Lógica de negocio en la base de datos 7
9. Requerimientos 8
10. Alcance del proyecto 10
11. Supuestos del proyecto 10
12. Metodología de gestión 11
13. Organización del equipo 12
14. Planificación 13
15. Gestión de riesgos 14
16. Control de versiones 15
17. Modelo BPSC (Biopsicosocial y cultural) 15
18. Historias de usuario 17
19. Casos de uso 19
20. Diagrama C4 21
21. Casos de aceptación 23
22. Conclusión 28
Referencias 28

1. Contexto del proyecto

BountyLine se plantea dentro del universo de un videojuego de mundo abierto ambientado en una ciudad futurista. En ese mundo los jugadores tienen conflictos entre sí: robos de créditos, ataques, deudas o zonas peligrosas que no pueden recorrer solos. BountyLine es la red social del juego donde los jugadores publican lo que les pasa, piden ayuda y contratan a otros jugadores que se dedican a resolver ese tipo de situaciones, los cazarrecompensas.

La plataforma maneja tres tipos de cuenta (Usuario, Cazarrecompensas y Administrador) y combina dos partes: un feed de publicaciones cortas, parecido a un microblog, y un mercado de misiones que se pagan con la moneda virtual del juego.

2. Problemática

Dentro del juego, un jugador puede querer recuperar créditos que le robaron, cobrarse una ofensa de otro jugador, conseguir protección para entrar a una zona peligrosa o simplemente delegar un conflicto que no quiere resolver por su cuenta. Hoy no existe un mecanismo centralizado y confiable que conecte a ese jugador con alguien dispuesto a resolver la situación a cambio de una recompensa. De esto se derivan cuatro problemas concretos:

- No hay un canal formal para pedir ayuda; todo depende de mensajes informales entre conocidos.
- No hay forma de saber qué jugadores son confiables o buenos en cierto tipo de trabajo, porque no existe una reputación verificable.
- No hay trazabilidad de los pagos ni garantía de que la recompensa prometida llegue al cazarrecompensas, ni de que el usuario recupere su dinero si el trabajo no se hace.
- No hay un mecanismo de emparejamiento que sugiera al cazarrecompensas adecuado según la zona, la especialidad y la reputación.

Desde el punto de vista de datos, el problema exige dos tipos de información muy distintos: pagos, misiones y calificaciones, donde la integridad es crítica, y contenido social de alto volumen y estructura variable. Por eso el caso sirve para aplicar tanto una base de datos relacional con lógica de negocio como una base de datos documental.

3. Objetivo general

Diseñar e implementar BountyLine, una plataforma web que gestione interacciones sociales y misiones de recompensa en un entorno de videojuego futurista, mediante funcionalidades de publicación, asignación y seguimiento de misiones, administración de créditos virtuales y evaluación de la reputación de los participantes.

4. Objetivos específicos

- Analizar las necesidades mediante supuestos, actores, requerimientos funcionales y no funcionales e historias de usuario.

- Diseñar la solución mediante modelos UML, arquitectura orientada a servicios y patrones de diseño justificados.

- Implementar una aplicación web de demostración para gestionar cuentas, publicaciones, misiones, créditos virtuales, postulaciones, chat y reputación.

- Validar la solución mediante pruebas de aceptación, trazabilidad, gestión de riesgos y documentación.

5. Descripción de la solución propuesta

5.1 Roles de usuario

- **Usuario:** publica en el feed, crea misiones con una recompensa, revisa sugerencias de cazarrecompensas, asigna la misión, conversa por chat, confirma el cierre y califica el servicio.
- **Cazarrecompensas:** gestiona su perfil, zona base y especialidades, consulta misiones disponibles, se postula, conversa con el usuario y recibe el pago al cierre de la misión.
- **Administrador:** administra los catálogos de zonas y especialidades, atiende reportes, oculta contenido inapropiado, suspende cuentas y puede cancelar misiones en conflicto.

5.2 Funcionalidades principales

- Registro e inicio de sesión para los tres tipos de cuenta.
- Feed de publicaciones con etiquetas, reacciones y comentarios.
- Publicación de misiones con zona, especialidad requerida, descripción y monto en créditos.
- Billetera virtual con recarga simulada, retención del monto al publicar la misión, pago al cerrar y reembolso al cancelar.
- Emparejamiento híbrido: sugerencia automática de cazarrecompensas calculada en la base de datos según zona, especialidad y reputación, y búsqueda manual con filtros.
- Postulación de cazarrecompensas y asignación por parte del usuario.
- Chat de texto por misión entre el usuario y el cazarrecompensas asignado.
- Calificación de 1 a 5 al cerrar la misión y cálculo automático de la reputación.
- Reportes de contenido o de cuentas y panel básico de moderación.
- Consultas de reportes para el administrador (misiones por zona, créditos movidos, cazarrecompensas mejor calificados, etiquetas más usadas).

5.3 Flujo general de uso

- Paso 1. El usuario publica en el feed lo que le pasó.
- Paso 2. El usuario crea una misión con zona, especialidad y monto. El sistema valida que tenga saldo y retiene el monto.
- Paso 3. El sistema sugiere cazarrecompensas ordenados por puntaje de emparejamiento. El usuario también puede buscar con filtros.
- Paso 4. Los cazarrecompensas se postulan y el usuario asigna a uno. Se habilita el chat de la misión.
- Paso 5. El cazarrecompensas inicia la misión y conversa con el usuario mientras la resuelve.

- Paso 6. El usuario confirma el cierre. En una sola transacción se libera el monto retenido, se acredita al cazarrecompensas y se registran los movimientos.
- Paso 7. El usuario califica el servicio y la reputación del cazarrecompensas se actualiza automáticamente.

5.4 Arquitectura de la solución

La solución tiene tres capas que se ejecutan en una máquina virtual con Ubuntu Server 24.04 LTS:

- **Presentación:** interfaz web en HTML, CSS y Typescript servida por la aplicación.
- **Aplicación:** API en Spring Boot. Para MySQL usa JDBC y llama a los procedimientos almacenados; para MongoDB usa el driver oficial.
- **Datos:** MySQL 8.0 para la información transaccional y MongoDB para el contenido social y la bitácora.

La solución implementada utiliza Angular en la capa de presentación, Spring Boot en la capa de aplicación, MySQL 8.0 como base de datos relacional y MongoDB como base de datos documental.

6. Justificación del modelo de datos

La plataforma mezcla información estructurada y transaccional, donde un error deja saldos inconsistentes, con información social de alto volumen y forma variable. Por eso se usa un enfoque de persistencia políglota: cada tipo de dato va al motor que mejor responde a sus necesidades (Sadalage y Fowler, 2012).

6.1 Base de datos relacional (MySQL)

En MySQL se almacenan cuentas, perfiles, billeteras, catálogos, misiones, postulaciones, movimientos de créditos, calificaciones, reportes y la auditoría de cuentas. Estos datos tienen estructura fija, relaciones bien definidas y requieren las propiedades ACID. El caso más claro es el cierre de una misión: debitar el monto retenido al usuario, acreditarlo al cazarrecompensas, registrar dos movimientos y cambiar el estado de la misión tiene que ocurrir completo o no ocurrir, lo cual se garantiza con una transacción dentro de un procedimiento almacenado.

6.2 Base de datos NoSQL (MongoDB)

En MongoDB se almacenan tres colecciones:

- **publicaciones:** cada documento guarda el texto, el autor (id de cuenta en MySQL y alias), las etiquetas, la zona, una referencia opcional a una misión, el conteo de reacciones por tipo, los comentarios embebidos y el estado de moderación. Un post se lee siempre junto con sus comentarios, así que embeberlos evita varias consultas por cada elemento del feed.
- **mensajes_chat:** un documento por mensaje con el id de la misión, el emisor, el texto y la fecha, con un índice compuesto por misión y fecha. Se eligió un documento por mensaje en lugar de un arreglo embebido para no tener documentos que crecen sin límite.

- **bitacora_actividad:** eventos de uso como inicios de sesión, búsquedas con los filtros usados y visualizaciones de misiones. Cada tipo de evento trae campos distintos, lo que encaja con un esquema flexible.

Este contenido se escribe y se lee con mucha frecuencia, cambia de forma con facilidad (nuevos tipos de reacción o de evento) y no requiere transacciones entre varias entidades. MongoDB permite agregar campos sin migraciones y resolver con su framework de agregación consultas como las etiquetas más usadas por zona o la actividad por semana.

Operaciones previstas en MongoDB:

- CRUD de publicaciones, comentarios, mensajes y eventos.
- Consultas complejas con filtros combinados (etiquetas, zona, rango de fechas, texto) y proyecciones.
- Agregaciones con $match, $unwind, $group, $sort y $lookup: etiquetas más usadas por zona, publicaciones por semana, cantidad promedio de mensajes por misión y filtros de búsqueda más usados.

7. Entidades principales identificadas (modelo preliminar)

El modelo relacional preliminar tiene 14 entidades. Cuenta es una superclase con especialización total y disjunta en Usuario, Cazarrecompensas y Administrador, por lo que se elaborará también el modelo Entidad-Relación extendido. El detalle de atributos se presentará en los modelos y en el diccionario de datos de la Fase 1.

Entidad	**Descripción**	**Relaciones principales**
Cuenta	Datos de acceso comunes: correo, alias (nombre del jugador en el juego), hash de la contraseña, rol, estado y fecha de registro.	1:1 con Usuario, Cazarrecompensas o Administrador; 1:1 con Billetera; 1:N con Auditoría_Cuenta; 1:N con Reporte
Usuario	Perfil del jugador que publica misiones y contrata cazarrecompensas.	1:N con Misión
Cazarrecompensas	Perfil del jugador que resuelve misiones; incluye zona base y reputación.	N:1 con Zona; N:M con Especialidad; 1:N con Misión (asignadas); N:M con Misión vía Postulación
Administrador	Perfil con permisos de moderación y gestión de catálogos.	1:N con Reporte (los atiende); 1:N con Zona y con Especialidad (las registra); 1:N con Misión (las que cancela)

Entidad	Descripción	Relaciones principales
Billetera	Saldo disponible y saldo retenido en créditos de cada cuenta de jugador.	1:1 con Cuenta; 1:N con Movimiento
Zona	Distrito del mundo del juego (centro, puerto, zona industrial, suburbios).	1:N con Misión; 1:N con Cazarrecompensas
Especialidad	Tipo de trabajo: recuperación de bienes, protección, escolta, venganza, sabotaje.	1:N con Misión; N:M con Cazarrecompensas
Cazarrecompensas_Especialidad	Entidad asociativa con el nivel de experiencia del cazarrecompensas en cada especialidad.	Resuelve la N:M entre Cazarrecompensas y Especialidad
Misión	Encargo publicado por un usuario: título, descripción, monto, estado, fechas y cazarrecompensas asignado.	N:1 con Usuario, Zona y Especialidad; N:1 opcional con Cazarrecompensas asignado; 1:1 con Calificación; 1:N con Movimiento
Postulación	Registro de cada cazarrecompensas que aplica a una misión, con fecha, mensaje y estado.	Resuelve la N:M entre Misión y Cazarrecompensas
Movimiento	Movimiento de créditos: recarga, retención, pago, cobro o reembolso, con monto y fecha.	N:1 con Billetera; N:1 opcional con Misión
Calificación	Puntaje de 1 a 5 y comentario del usuario sobre el cazarrecompensas.	1:1 con Misión cerrada
Auditoría_Cuenta	Historial de cambios de estado y de correo de las cuentas, llenado por un trigger.	N:1 con Cuenta
Reporte	Denuncia de una cuenta o de un contenido de MongoDB (guarda su id), con motivo y estado.	N:1 con Cuenta que reporta; N:1 opcional con Cuenta reportada; N:1 opcional con Administrador que lo atiende

Con este modelo se cubren las relaciones 1:1 (Cuenta con su perfil y con Billetera, Misión con Calificación), 1:N (Usuario con Misión, Zona con Misión, Billetera con Movimiento, Cuenta con Auditoría_Cuenta) y N:M (Cazarrecompensas con Especialidad y con Misión).

8. Lógica de negocio en la base de datos

8.1 Procedimientos almacenados

- sp_registrar_cuenta: crea la cuenta, el perfil según el rol y, si la cuenta es de Usuario o de Cazarrecompensas, su billetera, todo en una sola transacción.
- sp_recargar_saldo: acredita una recarga simulada y registra el movimiento.
- sp_publicar_mision: valida el saldo, crea la misión y mueve el monto de saldo disponible a saldo retenido.
- sp_postular: registra la postulación validando que la misión esté PUBLICADA y que el cazarrecompensas no esté postulado ya.
- sp_asignar_cazarrecompensas: asigna la misión, rechaza las demás postulaciones y cambia el estado a ASIGNADA.
- sp_cerrar_mision: libera el monto retenido, paga al cazarrecompensas, registra los movimientos y cierra la misión.
- sp_cancelar_mision: devuelve el monto retenido al usuario y cancela la misión.
- sp_sugerir_cazarrecompensas: devuelve los cazarrecompensas ordenados por puntaje de emparejamiento para una misión.
- sp_calificar_mision: registra la calificación de una misión cerrada.

8.2 Funciones

- fn_saldo_disponible: retorna el saldo disponible de una billetera.
- fn_reputacion: retorna el promedio de calificaciones de un cazarrecompensas.
- fn_puntaje_match: calcula el puntaje entre un cazarrecompensas y una misión a partir de coincidencia de zona, nivel en la especialidad y reputación.
- fn_misiones_completadas: retorna la cantidad de misiones cerradas por un cazarrecompensas.

8.3 Triggers

- trg_validar_estado_mision (BEFORE UPDATE en Misión): impide transiciones de estado no permitidas.
- trg_actualizar_reputacion (AFTER INSERT en Calificación): recalcula la reputación del cazarrecompensas.
- trg_validar_saldo (BEFORE UPDATE en Billetera): impide saldos negativos.
- trg_validar_calificacion (BEFORE INSERT en Calificación): solo permite calificar misiones CERRADAS y por su creador.
- trg_auditoria_cuenta (AFTER UPDATE en Cuenta): registra en Auditoría_Cuenta los cambios de estado y de correo, con el valor anterior, el valor nuevo, la fecha y el usuario de base de datos que hizo el cambio.

8.4 Transacciones, excepciones y seguridad

- Los procedimientos que mueven créditos usan START TRANSACTION, COMMIT y ROLLBACK, con un DECLARE EXIT HANDLER FOR SQLEXCEPTION que revierte todo ante cualquier error.
- Las reglas de negocio incumplidas (saldo insuficiente, misión en estado inválido, postulación duplicada) se rechazan con SIGNAL SQLSTATE '45000' y un mensaje claro que la aplicación muestra al usuario.
- La integridad se asegura con llaves primarias y foráneas, restricciones UNIQUE, NOT NULL y CHECK (montos positivos, calificación entre 1 y 5, estados válidos).
- Se crean usuarios de base de datos con privilegios mínimos: la aplicación solo tiene permiso de ejecutar procedimientos y consultar vistas, no de modificar tablas directamente. Un usuario administrador separado se usa para mantenimiento.
- Las contraseñas se guardan con hash (BCrypt) generado en la aplicación; nunca en texto plano.

9. Requerimientos

La prioridad se asigna con la técnica MoSCoW: Debe (indispensable para la entrega), Debería (importante pero no bloquea la entrega) y Podría (se hace si queda tiempo).

9.1 Requerimientos funcionales

ID	Requerimiento	Prioridad	Trazabilidad
RF-01	Registrar cuentas de Usuario y Cazarrecompensas e iniciar sesión. Las cuentas de Administrador se crean mediante el mecanismo administrativo definido para el sistema.	Debe	Cuenta, Billetera, sp_registrar_cuenta
RF-02	Gestionar el perfil del cazarrecompensas, su zona base y especialidades.	Debe	Cazarrecompensas, Cazarrecompensas_Especialidad
RF-03	Recargar créditos de forma simulada y consultar el saldo y los movimientos.	Debe	Billetera, Movimiento, sp_recargar_saldo
RF-04	Publicar misiones con zona, especialidad y monto, reteniendo el monto.	Debe	Misión, sp_publicar_mision
RF-05	Sugerir cazarrecompensas para una misión según puntaje de emparejamiento.	Debe	sp_sugerir_cazarrecompensas, fn_puntaje_match
RF-06	Buscar cazarrecompensas y misiones con filtros manuales.	Debería	Vistas de consulta, bitacora_actividad

ID	Requerimiento	Prioridad	Trazabilidad
RF-07	Postularse a una misión y asignar un cazarrecompensas.	Debe	Postulación, sp_postular, sp_asignar_cazarrecompensas
RF-08	Cerrar una misión con pago automático o cancelarla con reembolso.	Debe	sp_cerrar_mision, sp_cancelar_mision
RF-09	Calificar una misión cerrada y actualizar la reputación.	Debe	Calificación, trg_actualizar_reputacion
RF-10	Publicar en el feed, comentar y reaccionar.	Debe	Colección publicaciones
RF-11	Enviar y consultar mensajes de chat por misión.	Debe	Colección mensajes_chat
RF-12	Reportar cuentas o contenido.	Debería	Reporte
RF-13	Moderar: ocultar contenido, suspender cuentas y gestionar catálogos.	Debería	Reporte, Zona, Especialidad, publicaciones
RF-14	Generar reportes de misiones, créditos y actividad para el administrador.	Debería	Documento SQL de reportes, agregaciones MongoDB
RF-15	Registrar la actividad de los usuarios en una bitácora.	Podría	Colección bitacora_actividad

9.2 Requerimientos no funcionales

ID	Requerimiento	Prioridad	Trazabilidad
RNF-01	Integridad: ningún movimiento de créditos puede quedar a medias; toda operación de pago es atómica.	Debe	Transacciones en procedimientos
RNF-02	Seguridad: contraseñas con hash y usuarios de base de datos con privilegios mínimos.	Debe	Usuarios y GRANT de MySQL
RNF-03	Normalización del modelo relacional hasta 3FN.	Debe	Modelo relacional, diccionario de datos
RNF-04	Rendimiento: el feed y las sugerencias responden en menos de 2 segundos con los datos de prueba.	Debería	Índices en MySQL y MongoDB
RNF-05	Portabilidad: toda la solución se instala en una máquina virtual Linux siguiendo una guía documentada.	Debe	Documento de instalación
RNF-06	Mantenibilidad: scripts de creación, inserción y lógica versionados y ejecutables en orden.	Debe	Repositorio GitHub

ID	Requerimiento	Prioridad	Trazabilidad
RNF-07	Usabilidad: interfaz sencilla en español con mensajes de error comprensibles.	Debería	Aplicación web

La trazabilidad se mantendrá en una matriz que relaciona cada requerimiento con su elemento del modelo, su script, la funcionalidad de la aplicación y la prueba que lo valida.

10. Alcance del proyecto

Dentro del alcance	Fuera del alcance
Registro e inicio de sesión de los tres tipos de cuenta.	Videojuego o motor gráfico; solo se construye la plataforma web.
Feed de publicaciones con comentarios y reacciones.	Pasarela de pago real; los créditos son simulados.
Billetera con recarga simulada, retención, pago y reembolso.	Aplicación móvil nativa.
Sugerencia automática de cazarrecompensas y búsqueda con filtros.	Mapa interactivo; las zonas son un catálogo.
Publicación, postulación, asignación, cierre y cancelación de misiones.	Chat en tiempo real con WebSockets.
Chat de texto por misión.	Carga de imágenes o videos.
Calificaciones y reputación.	Sistema de niveles o experiencia tipo RPG.
Panel básico de moderación y reportes para el administrador.	Analítica avanzada o tableros de BI.
Despliegue en máquina virtual Linux, repositorio Git y video demostrativo.	Despliegue en la nube o en un dominio público.

11. Supuestos del proyecto

Los supuestos completos, numerados y organizados por entidad, están en el documento de supuestos que acompaña esta propuesta. Los que más influyen en el diseño del modelo son:

- BountyLine es una plataforma web que funciona como red social dentro de un videojuego de mundo abierto ambientado en una ciudad futurista. No se desarrolla el videojuego ni ningún motor gráfico. (S1)
- La moneda de la plataforma son los créditos, una moneda virtual del juego sin equivalencia con dinero real. No existe integración con pasarelas de pago. (S3)
- Existen tres roles: Usuario, Cazarrecompensas y Administrador. Cada cuenta tiene exactamente uno de ellos. (S12)
- Cada cazarrecompensas opera en exactamente una zona base, y en una zona pueden operar varios cazarrecompensas. (S25)

- Cada cazarrecompensas registra al menos una especialidad y, para cada una, un nivel de experiencia de 1 a 5. Una especialidad puede ser dominada por varios cazarrecompensas. (S26)
- El administrador gestiona los catálogos de zonas y especialidades. Cada zona y cada especialidad queda asociada al administrador que la registró, y un administrador puede registrar varias. (S32)
- Toda cuenta de Usuario y de Cazarrecompensas tiene exactamente una billetera, que se crea en la misma transacción del registro. Las cuentas de Administrador no tienen billetera. (S37)
- Al publicar una misión, el monto de la recompensa se retiene del saldo disponible del usuario. Si el saldo no alcanza, la misión no se crea. (S52)
- Los estados de una misión son PUBLICADA, ASIGNADA, EN_CURSO, CERRADA y CANCELADA. Las transiciones permitidas son PUBLICADA a ASIGNADA, ASIGNADA a EN_CURSO, EN_CURSO a CERRADA, y de PUBLICADA, ASIGNADA o EN_CURSO a CANCELADA. (S55)
- Varios cazarrecompensas pueden postularse a una misión, pero solo uno queda asignado, y el asignado debe ser uno de los que se postularon. (S56)
- El pago se libera cuando el usuario confirma el cierre de la misión. En una sola transacción se descuenta el monto retenido, se acredita al cazarrecompensas, se registran los movimientos y se guarda la fecha de cierre. (S59)
- Cada misión cerrada admite como máximo una calificación, y cada calificación corresponde a una sola misión. (S63)
- Un reporte se dirige a una cuenta o a una publicación o comentario de MongoDB, nunca a ambos al mismo tiempo. Cuando se dirige a un contenido, se guarda el identificador del documento de MongoDB. (S72)
- Los cambios de estado y de correo de una cuenta quedan registrados automáticamente en la auditoría mediante un trigger. (S76)

12. Metodología de gestión

El proyecto se gestiona con Scrum apoyado en un tablero Kanban, es decir, un enfoque híbrido. Se eligió Scrum porque el trabajo se puede dividir en incrementos que se revisan con el docente (Fase 1 y Fase 2) y porque el modelo de datos probablemente tendrá ajustes después de las primeras revisiones; con sprints cortos esos cambios se incorporan sin rehacer todo el proyecto (Schwaber y Sutherland, 2020). El tablero Kanban en GitHub Projects, con columnas Por hacer, En progreso, En revisión y Hecho, sirve para ver el avance diario y limitar el trabajo en curso. Cascada no se eligió porque obliga a cerrar el diseño antes de implementar, y Kanban puro no da los cortes de entrega que pide el curso.

- Sprints de dos semanas.
- Product backlog con los requerimientos de la sección 9, priorizados con MoSCoW.
- Planeación al inicio de cada sprint, seguimiento corto dos veces por semana, revisión con entregable al final del sprint y retrospectiva.

- El docente actúa como cliente; el Product Owner del equipo traduce sus observaciones en ítems del backlog.

13. Organización del equipo

13.1 Roles

Integrante	Rol y responsabilidades
Julian Samuel Rodriguez Hernandez	Scrum Master y responsable de la base de datos relacional: modelos E-R, normalización, scripts de creación y lógica en MySQL.
Lina Patiño	Product Owner y responsable de la aplicación web: backlog, requerimientos, interfaz, integración con los procedimientos y pruebas.
Gabriella Castro	Responsable de MongoDB e infraestructura: modelo documental, consultas y agregaciones, máquina virtual e instalación de los motores.

Aunque cada integrante lidera un frente, los tres participan en el modelado, la programación y la documentación, y cada uno debe tener commits propios en todos los componentes principales.

13.2 Matriz RACI

R: responsable de ejecutar. A: aprueba y responde por el resultado. C: consultado. I: informado.

Actividad	Julian Rodriguez	Lina Patiño	Gabriella Castro
Propuesta y documento de supuestos	R	A	R
Modelo E-R (Chen) y E-R extendido	A/R	C	R
Modelo relacional y normalización 3FN	A/R	C	C
Diccionario de datos	R	C	A/R
Documento SQL de reportes	R	A/R	C
Análisis de SO e implementación de la VM	C	I	A/R
Análisis de RDBMS e implementación	A/R	C	R
Scripts de creación, inserción y varios	A/R	R	C
Procedimientos, funciones y triggers	A/R	R	C
Modelo documental e implementación MongoDB	C	R	A/R
Aplicación web y conexiones a los dos motores	R	A/R	R
Gestión del repositorio Git	A	R	R

Actividad	Julian Rodriguez	Lina Patiño	Gabriella Castro
Cronograma, riesgos y seguimiento Scrum	A/R	C	C
Informe ejecutivo final	R	A/R	R
Video demostrativo	R	R	A/R

14. Planificación

14.1 Cronograma preliminar

El cronograma se expresa en semanas porque las fechas exactas dependen del calendario del docente. El diagrama de Gantt detallado se elaborará en GanttProject a partir de esta tabla.

Actividad	Sprint	Semanas	Horas estimadas	Entregable
Propuesta, supuestos y backlog inicial	1	1	12	Propuesta y documento de supuestos
Modelos E-R (Chen y extendido)	1	1 a 2	14	Modelos E-R
Modelo relacional, 3FN y diccionario de datos	1	2	16	Modelo relacional y diccionario
Documento SQL de reportes	1	2	8	Documento SQL de reportes
Análisis de SO e instalación de la VM	2	3	10	Análisis de SO y VM
Análisis de RDBMS, instalación y scripts de creación e inserción	2	3 a 4	18	Scripts de creación e inserción
Procedimientos, funciones, triggers y seguridad	2	4	24	Scripts varios
Modelo documental e implementación MongoDB	3	5	16	Modelo documental y scripts MongoDB
Aplicación web y conexión con los dos motores	3	5 a 6	30	Aplicación funcional
Pruebas integrales y correcciones	4	7	14	Evidencia de pruebas
Informe ejecutivo, video y sustentación	4	8	16	Informe, video y repositorio final

El total estimado es de 178 horas, unas 60 horas por integrante, dentro de la disponibilidad de 8 horas semanales durante 8 semanas.

14.2 Hitos

- H1, semana 1: propuesta y supuestos entregados.
- H2, semana 2: entrega de la Fase 1 (modelos, diccionario de datos y documento SQL de reportes).
- H3, semana 4: máquina virtual con MySQL instalado y lógica de negocio funcionando.
- H4, semana 6: aplicación web conectada a MySQL y a MongoDB.
- H5, semana 8: entrega final de la Fase 2 con informe ejecutivo, repositorio y video.

14.3 Entregables

- **Fase 1:** documento de propuesta, documento de supuestos, modelo E-R en notación de Chen, modelo E-R extendido, modelo E-R y modelo relacional en notación Pata de Gallina (MySQL Workbench), diccionario de datos y documento SQL de reportes.
- **Fase 2:** análisis de selección de SO, implementación de la VM, análisis de selección de RDBMS, implementación del RDBMS, scripts de creación, inserción y varios, modelo documental MongoDB, implementación MongoDB, conexión de la aplicación con ambos motores, repositorio Git y video demostrativo de máximo 15 minutos.
- **Cierre:** informe ejecutivo de máximo 5 páginas sin anexos (problema, solución, arquitectura, decisiones técnicas, resultados, lecciones aprendidas y conclusiones).

15. Gestión de riesgos

La probabilidad (P) y el impacto (I) se califican de 1 (bajo) a 3 (alto). El nivel es P por I: de 1 a 2 es bajo, de 3 a 4 es medio y de 6 a 9 es alto.

ID	Riesgo	P	I	Nivel	Estrategia de mitigación
R1	Errores en el modelo de datos detectados tarde que obligan a rehacer scripts.	2	3	6 Alto	Revisar el modelo con el docente en la Fase 1 y validar la 3FN antes de escribir scripts.
R2	Incompatibilidad o problemas de instalación de MySQL o MongoDB en la VM.	2	2	4 Medio	Instalar la VM en la semana 3, documentar cada paso y exportar una imagen OVA funcional como respaldo.
R3	Retrasos por carga académica de otras materias.	3	2	6 Alto	Holgura en las semanas 7 y 8, seguimiento dos veces por semana y priorización MoSCoW del backlog.
R4	Distribución desigual del trabajo y commits concentrados en una persona.	2	3	6 Alto	Matriz RACI, tareas asignadas en el tablero y revisión semanal de commits por integrante.
R5	Inconsistencias entre MySQL y MongoDB (referencias a cuentas o misiones que no existen).	2	2	4 Medio	Validar en la aplicación que el id exista en MySQL antes de escribir en MongoDB y usar estados lógicos en lugar de borrados.

ID	Riesgo	P	I	Nivel	Estrategia de mitigación
R6	Pérdida de código o conflictos de integración en el repositorio.	1	3	3 Medio	Ramas por funcionalidad, pull requests revisados y rama main protegida.
R7	Fallas en la lógica de pago que dejen saldos inconsistentes.	2	3	6 Alto	Transacciones con manejador de excepciones, trigger de saldo no negativo y casos de prueba para cada procedimiento.

Matriz de probabilidad contra impacto:

Probabilidad / Impacto	Impacto bajo (1)	Impacto medio (2)	Impacto alto (3)
Alta (3)		R3	
Media (2)		R2, R5	R1, R4, R7
Baja (1)			R6

16. Control de versiones

- Repositorio en GitHub con los tres integrantes como colaboradores.
- Ramas: main (versiones entregadas), develop (integración) y ramas feature/nombre-funcionalidad para cada tarea.
- Integración a develop mediante pull request revisado por otro integrante.
- Commits descriptivos con prefijo de tipo, por ejemplo feat: procedimiento sp_cerrar_mision o docs: diccionario de datos de Misión.
- Estructura de carpetas: docs, modelos, sql (creacion, insercion, logica, reportes), mongo, app y vm.
- Etiquetas (tags) para la entrega de cada fase, de modo que el historial muestre la evolución del proyecto.

17. Modelo BPSC (Biopsicosocial y cultural)

El modelo biopsicosocial y cultural permite analizar BountyLine desde las dimensiones individuales, sociales y culturales de sus participantes, considerando cómo interactúan dentro de una comunidad digital ambientada en un mundo futurista.

17.1 Dimensión biológica

En el contexto de BountyLine, esta dimensión se relaciona con las condiciones físicas de los personajes dentro del videojuego y con las situaciones que motivan a los jugadores a solicitar ayuda.

Por ejemplo, un personaje puede necesitar protección para atravesar una zona peligrosa o acompañamiento durante una misión. Estas situaciones constituyen parte del contexto narrativo del sistema y motivan la publicación de encargos.

Relación con el sistema: las zonas, las especialidades y las misiones permiten representar las necesidades operativas de los personajes dentro del universo ficticio.

17.2 Dimensión psicológica

Esta dimensión comprende aspectos como la confianza, la percepción de seguridad, la toma de decisiones y la reputación de los participantes.

BountyLine incorpora mecanismos que ayudan a reducir la incertidumbre al momento de contratar un servicio. Entre ellos se encuentran las calificaciones, el historial de misiones completadas y las sugerencias de cazarrecompensas.

Relación con el sistema: las entidades Calificación, Misión y Cazarrecompensas, junto con la función fn_reputacion, permiten representar y consultar indicadores de confianza entre los participantes.

17.3 Dimensión social

Esta dimensión aborda las interacciones entre usuarios, cazarrecompensas y administradores.

BountyLine facilita la publicación de experiencias, la solicitud de ayuda, la búsqueda de participantes, la postulación a misiones y la comunicación mediante un chat asociado a cada encargo.

Relación con el sistema: el feed, los comentarios, las reacciones, las postulaciones y los mensajes permiten representar las interacciones sociales de la plataforma.

17.4 Dimensión cultural

Esta dimensión se relaciona con las normas, los valores y las prácticas propias de la comunidad ficticia del videojuego.

En BountyLine, los participantes utilizan créditos virtuales, reconocen especialidades, valoran la reputación de los cazarrecompensas y siguen reglas para publicar, asignar y cerrar misiones.

Relación con el sistema: los catálogos de zonas y especialidades, los estados de las misiones, las reglas de negocio y las políticas de moderación establecen las condiciones de participación dentro de la plataforma.

17.5 Integración del modelo BPSC con BountyLine

Dimensión	Aplicación en BountyLine	Elementos relacionados
Biológica	Necesidades de protección y desplazamiento de los personajes.	Zona, Especialidad, Misión.

Dimensión	Aplicación en BountyLine	Elementos relacionados
Psicológica	Confianza y evaluación de los servicios.	Calificación, Cazarrecompensas, reputación.
Social	Interacción, comunicación y colaboración entre jugadores.	Publicaciones, comentarios, postulaciones y chat.
Cultural	Normas, créditos virtuales y prácticas de la comunidad.	Billetera, Movimiento, estados de misión y moderación.

18. Historias de usuario

18.1 Historia de usuario principales

ID	Historia de usuario	Prioridad	Requerimiento relacionado
HU-01	Como usuario, quiero registrar una cuenta para acceder a las funcionalidades de BountyLine.	Debe	RF-01
HU-02	Como cazarrecompensas, quiero registrar mi zona base y mis especialidades para recibir sugerencias de misiones adecuadas.	Debe	RF-02, RF-05
HU-03	Como usuario, quiero recargar créditos virtuales para disponer de saldo con el que pueda publicar misiones.	Debe	RF-03
HU-04	Como usuario, quiero publicar una misión indicando su zona, especialidad, descripción y recompensa para solicitar ayuda.	Debe	RF-04
HU-05	Como usuario, quiero recibir sugerencias de cazarrecompensas según su zona, especialidad y reputación para seleccionar un candidato apropiado.	Debe	RF-05
HU-06	Como cazarrecompensas, quiero postularme a misiones publicadas para ofrecer mis servicios.	Debe	RF-07
HU-07	Como usuario, quiero asignar un cazarrecompensas entre quienes se postularon para iniciar la ejecución de la misión.	Debe	RF-07
HU-08	Como usuario, quiero confirmar el cierre de una misión para que se realice el pago de la recompensa.	Debe	RF-08
HU-09	Como usuario, quiero cancelar una misión cuando corresponda para recuperar el monto retenido de acuerdo con las reglas del sistema.	Debe	RF-08

ID	Historia de usuario	Prioridad	Requerimiento relacionado
HU-10	Como usuario, quiero calificar al cazarrecompensas después de finalizar una misión para contribuir a su reputación.	Debe	RF-09
HU-11	Como usuario, quiero publicar contenido, comentar y reaccionar para compartir experiencias con otros jugadores.	Debe	RF-10
HU-12	Como usuario o cazarrecompensas asignado, quiero intercambiar mensajes asociados a una misión para coordinar su ejecución.	Debe	RF-11
HU-13	Como usuario, quiero reportar cuentas o contenido inapropiado para solicitar la intervención del administrador.	Debería	RF-12
HU-14	Como administrador, quiero moderar contenido y gestionar reportes para mantener el funcionamiento de la comunidad.	Debería	RF-13
HU-15	Como administrador, quiero consultar reportes de misiones, créditos y actividad para conocer el comportamiento de la plataforma.	Debería	RF-14

18.2 Criterios de aceptación

Los siguientes criterios pueden utilizarse para validar las historias principales. Los casos de aceptación detallados se desarrollarán en la sección 21.

HU-03. Recarga de créditos

- Dado que un usuario tiene una billetera registrada, cuando solicita una recarga simulada con un monto válido, el sistema incrementa su saldo disponible.

- El sistema registra el movimiento correspondiente.

- Si el monto no es válido, la operación se rechaza y no modifica el saldo.

HU-04. Publicación de una misión

- Dado que el usuario tiene saldo suficiente, cuando publica una misión con los datos obligatorios, el sistema registra la misión y retiene el monto de la recompensa.

- Si el saldo disponible es insuficiente, la misión no se crea y el saldo permanece sin cambios.

- La operación debe evitar que el monto quede retenido sin que exista la misión correspondiente.

HU-06. Postulación a una misión

- Dada una misión en estado PUBLICADA, un cazarrecompensas puede registrar su postulación.

- Si intenta postularse nuevamente a la misma misión, el sistema rechaza la duplicación.

- Si la misión no está disponible para postulaciones, el sistema rechaza la operación.

HU-07. Asignación de un cazarrecompensas

- El usuario puede asignar únicamente a un cazarrecompensas que se haya postulado a la misión.
- Al realizar la asignación, la misión cambia a ASIGNADA.
- Las demás postulaciones dejan de estar disponibles para asignación.

HU-08. Cierre y pago de una misión

- El sistema permite cerrar una misión que haya alcanzado el estado requerido para el cierre.
- El pago al cazarrecompensas, la actualización de la misión y el registro de los movimientos se realizan dentro de una única transacción.
- Si ocurre un error, la operación se revierte para evitar inconsistencias en los créditos.

HU-10. Calificación

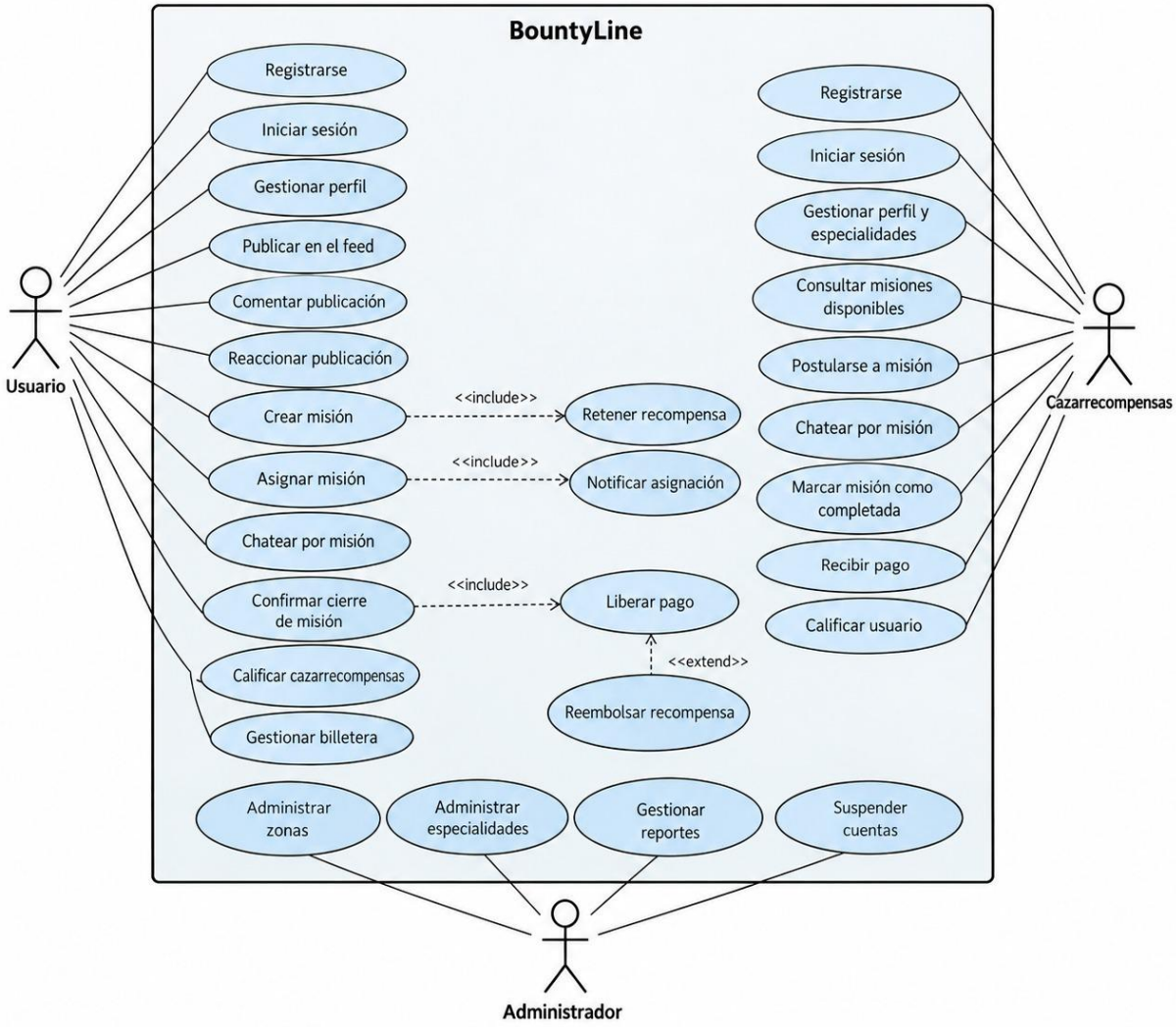
- El usuario creador de la misión puede calificarla después de su cierre.
- La calificación debe ser un valor entero entre 1 y 5.
- El sistema no permite más de una calificación por misión.
- La reputación se actualiza conforme a las calificaciones registradas.

HU-12. Chat por misión

- Los mensajes quedan asociados a la misión correspondiente.
- Solo los participantes autorizados pueden acceder al chat de la misión.
- Cada mensaje registra el emisor, el contenido y la fecha.

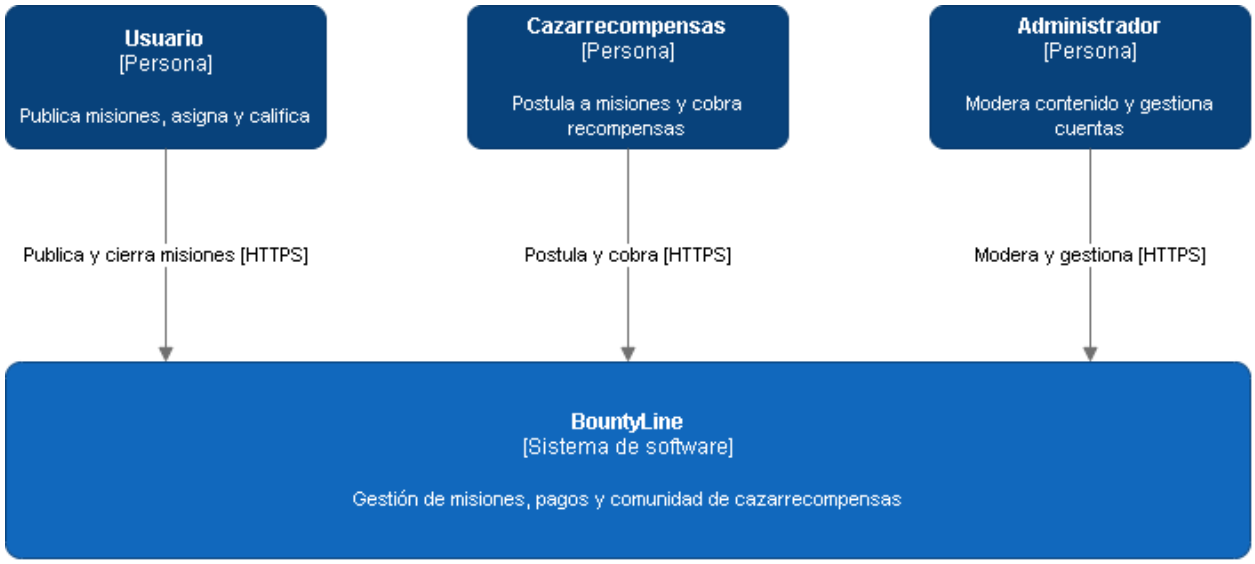
19. Casos de uso

Diagrama de casos de uso - BountyLine

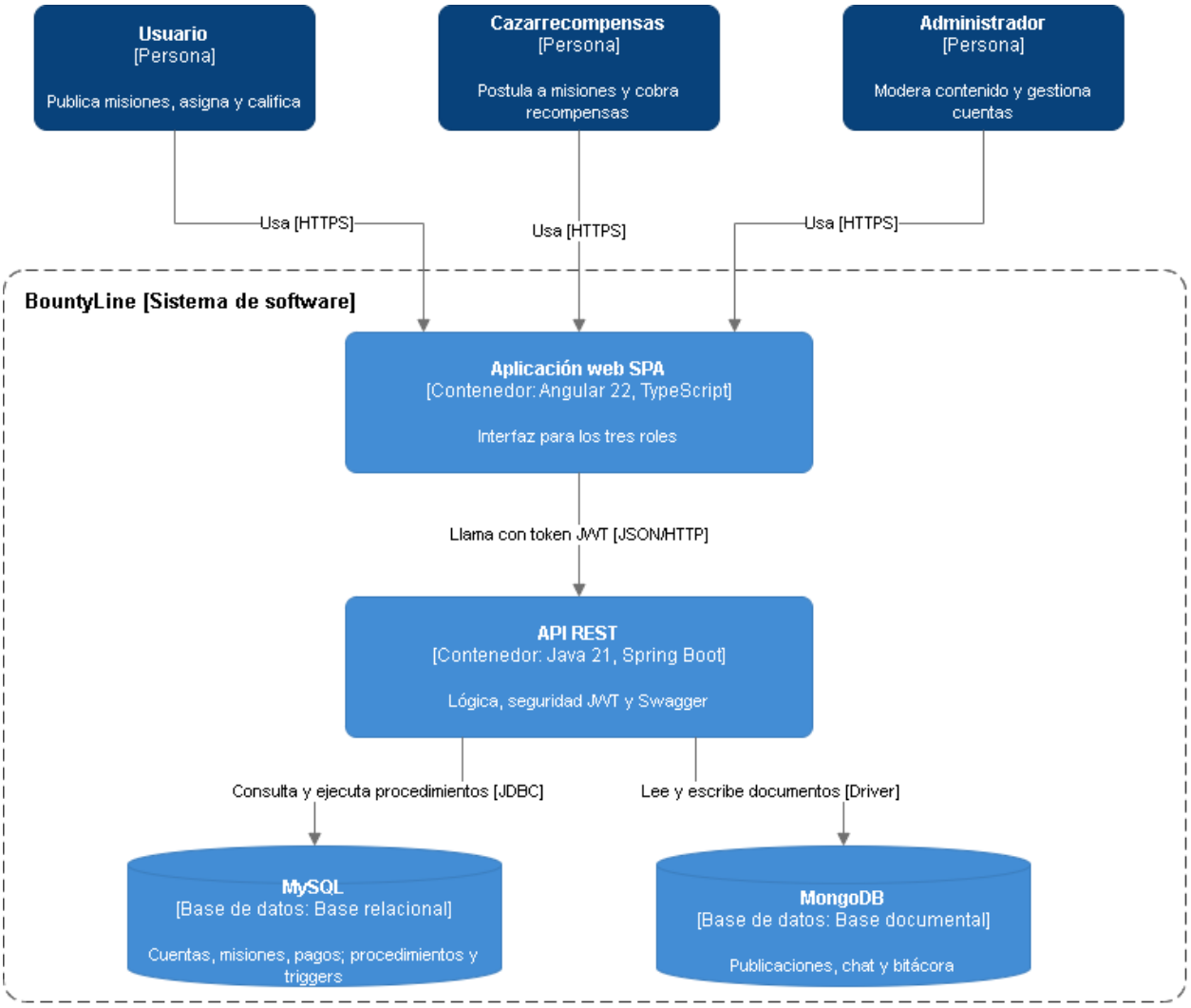


20. Diagrama C4

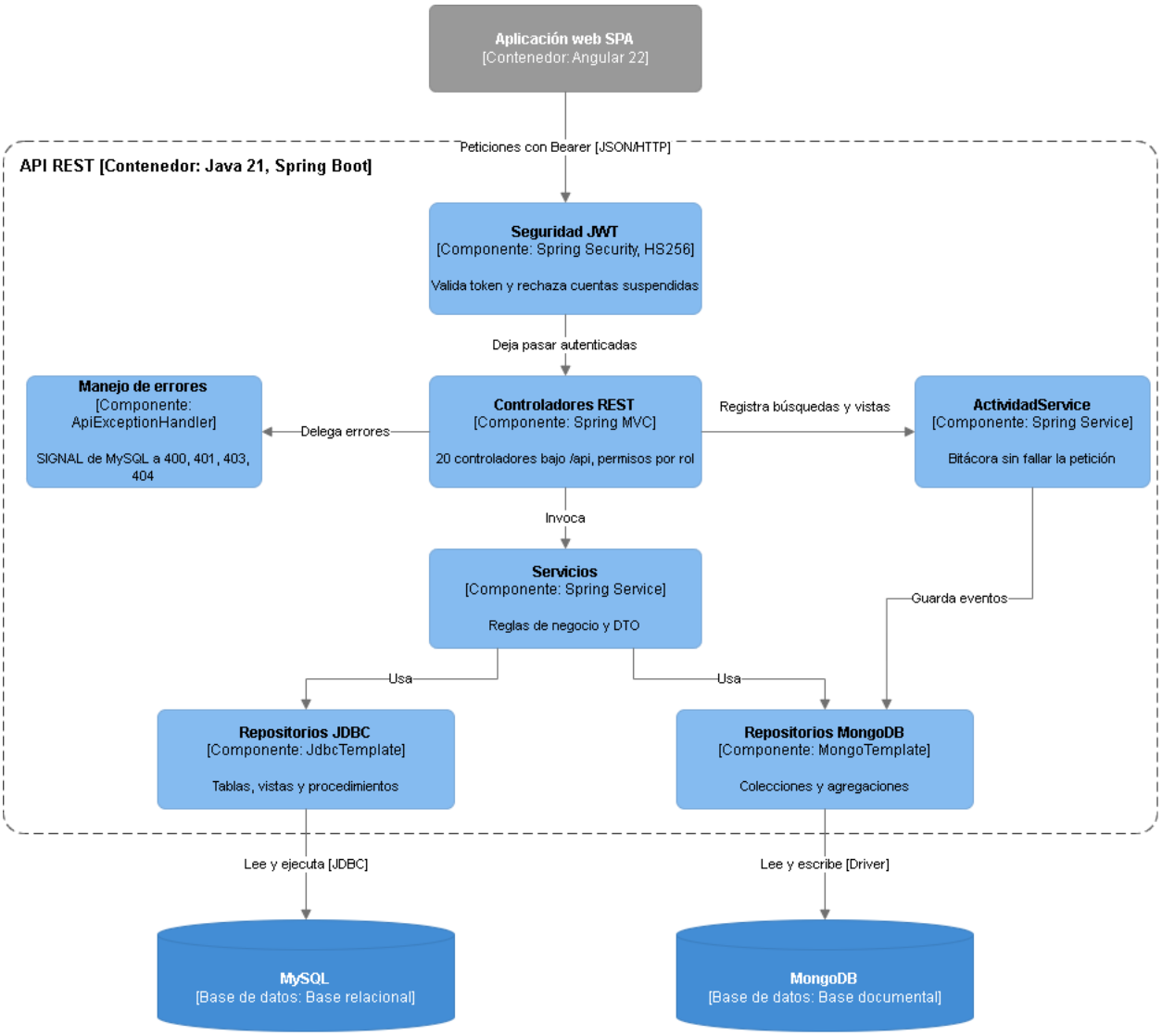
20.1 C4 nivel 1: contexto



20.2 C4 nivel 2: contenedores



20.3 C4 nivel 3: componentes (reducido, versión completa en PDF)



21. Casos de aceptación

ID	Caso de aceptación	Requerimiento relacionado	Precondiciones	Pasos de ejecución	Resultado esperado
CA-01	Registro de cuenta	RF-01	El usuario no tiene una cuenta registrada con ese correo o alias.	1. Ingresar los datos requeridos. 2. Seleccionar el tipo de cuenta.	El sistema registra la cuenta y crea el perfil correspondiente. Si aplica, crea también su billetera virtual.

ID	Caso de aceptación	Requerimiento relacionado	Precondiciones	Pasos de ejecución	Resultado esperado
				3. Enviar el formulario.	
CA-02	Inicio de sesión	RF-01	El usuario tiene una cuenta registrada y activa.	1. Ingresar correo y contraseña. 2. Enviar los datos.	El sistema valida las credenciales y permite el acceso. Si son incorrectas, rechaza el ingreso.
CA-03	Gestión del perfil del cazarrecompensas	RF-02	El usuario inició sesión con una cuenta de cazarrecompensas.	1. Abrir el perfil. 2. Registrar o modificar la zona base y las especialidades. 3. Guardar los cambios.	El sistema conserva la información del perfil y la utiliza en las búsquedas y sugerencias de misiones.
CA-04	Recarga de créditos virtuales	RF-03	El usuario tiene una cuenta y una billetera registrada.	1. Solicitar una recarga simulada. 2. Ingresar un monto válido. 3. Confirmar la operación.	El saldo disponible aumenta por el monto indicado y se registra el movimiento correspondiente.
CA-05	Rechazo de recarga inválida	RF-03	El usuario tiene una billetera registrada.	1. Solicitar una recarga. 2. Ingresar un monto inválido, cero o negativo. 3. Confirmar.	El sistema rechaza la operación y conserva el saldo y los movimientos anteriores sin cambios.
CA-06	Publicación de una misión	RF-04	El usuario inició sesión y dispone de saldo suficiente.	1. Ingresar título, descripción, zona, especialidad y recompensa.	El sistema registra la misión y retiene de la billetera del usuario el monto de la recompensa.

ID	Caso de aceptación	Requerimiento relacionado	Precondiciones	Pasos de ejecución	Resultado esperado
				2. Enviar la solicitud.	
CA-07	Publicación con saldo insuficiente	RF-04	El usuario tiene una billetera, pero no dispone del saldo necesario.	1. Crear una misión. 2. Indicar una recompensa superior al saldo disponible. 3. Intentar publicarla.	El sistema rechaza la publicación. No crea la misión ni modifica el saldo disponible o retenido.
CA-08	Sugerencia de cazarrecompensas	RF-05	Existe una misión publicada con zona y especialidad definidas, y hay perfiles de cazarrecompensas registrados.	1. Consultar las sugerencias de la misión. 2. Revisar los resultados.	El sistema presenta cazarrecompensas ordenados según el puntaje de coincidencia por zona, especialidad y reputación.
CA-09	Búsqueda manual de cazarrecompensas	RF-06	Existen cazarrecompensas registrados.	1. Abrir la búsqueda. 2. Aplicar filtros disponibles. 3. Consultar los resultados.	El sistema presenta los perfiles que coinciden con los filtros seleccionados.
CA-10	Postulación a una misión	RF-07	La misión está en estado PUBLICADA y el usuario inició sesión como cazarrecompensas.	1. Consultar las misiones disponibles. 2. Seleccionar una misión. 3. Enviar la postulación.	El sistema registra la postulación y la asocia con el cazarrecompensas y la misión correspondiente.
CA-11	Rechazo de postulación duplicada	RF-07	El cazarrecompensas ya se postuló a una misión.	1. Abrir nuevamente la misma misión. 2. Intentar	El sistema rechaza la postulación duplicada y

ID	Caso de aceptación	Requerimiento relacionado	Precondiciones	Pasos de ejecución	Resultado esperado
				postularse otra vez.	conserva el registro original.
CA-12	Asignación de un cazarrecompensas	RF-07	Existe una misión publicada y al menos un cazarrecompensas se ha postulado.	1. El usuario consulta las postulaciones. 2. Selecciona un candidato que se haya postulado. 3. Confirma la asignación.	El sistema asigna al cazarrecompensas seleccionado, cambia el estado de la misión a ASIGNADA y deshabilita las demás postulaciones para esa asignación.
CA-13	Rechazo de asignación no válida	RF-07	Existe una misión, pero el candidato seleccionado no tiene una postulación válida.	1. Intentar asignar a ese candidato. 2. Confirmar la operación.	El sistema rechaza la asignación y mantiene el estado de la misión y sus postulaciones sin cambios.
CA-14	Chat asociado a una misión	RF-11	La misión tiene un usuario creador y un cazarrecompensas asignado.	1. Ingresar al chat de la misión. 2. Enviar un mensaje. 3. Consultar los mensajes.	El sistema almacena el mensaje con el identificador de la misión, el emisor, el contenido y la fecha. Solo los participantes autorizados pueden acceder al chat.
CA-15	Cierre y pago de una misión	RF-08	La misión alcanzó el estado requerido para el cierre y tiene una recompensa retenida.	1. El usuario confirma el cierre. 2. El sistema procesa la operación. 3. Consultar el estado de la	La misión queda cerrada, el monto retenido se acredita al cazarrecompensas y se registran los movimientos correspondientes

ID	Caso de aceptación	Requerimiento relacionado	Precondiciones	Pasos de ejecución	Resultado esperado
				misión y los movimientos.	dentro de una única transacción.
CA-16	Reversión ante error durante el pago	RF-08	Existe una misión en proceso de cierre con una recompensa retenida.	1. Provocar o simular un error controlado durante la operación. 2. Consultar el estado de la misión y los saldos.	La transacción se revierte. No quedan pagos parciales ni saldos inconsistentes, y la misión no queda cerrada incorrectamente.
CA-17	Cancelación y reembolso de una misión	RF-08	Existe una misión que puede cancelarse según las reglas del sistema y tiene una recompensa retenida.	1. Solicitar la cancelación. 2. Confirmar la operación. 3. Consultar la billetera y el estado de la misión.	El sistema cancela la misión, devuelve el monto retenido al usuario y registra el movimiento de reembolso.
CA-18	Calificación de una misión cerrada	RF-09	La misión está cerrada y el usuario es quien la creó.	1. Abrir la misión cerrada. 2. Seleccionar una calificación entre 1 y 5. 3. Enviar la calificación.	El sistema registra la calificación y actualiza la reputación del cazarrecompensas.
CA-19	Rechazo de calificación inválida o duplicada	RF-09	Existe una misión cerrada.	1. Intentar registrar una calificación fuera del rango de 1 a 5, o enviar una segunda calificación	El sistema rechaza la calificación inválida o duplicada y conserva el registro válido existente.

ID	Caso de aceptación	Requerimiento relacionado	Precondiciones	Pasos de ejecución	Resultado esperado
				para la misma misión.	
CA-20	Publicaciones, comentarios y reacciones	RF-10	El usuario inició sesión y tiene permiso para participar en el feed.	1. Crear una publicación. 2. Agregar un comentario. 3. Reaccionar a una publicación.	El sistema almacena la publicación, el comentario y la reacción asociados con sus respectivos autores.
CA-21	Reporte de cuenta o contenido	RF-12	El usuario inició sesión y existe una cuenta o contenido que puede reportarse.	1. Seleccionar la cuenta o publicación. 2. Indicar el motivo. 3. Enviar el reporte.	El sistema registra el reporte con su motivo, autor y estado para su posterior revisión.
CA-22	Moderación y consultas administrativas	RF-13, RF-14	El administrador inició sesión con los permisos correspondientes.	1. Consultar los reportes. 2. Revisar un caso. 3. Ejecutar una acción de moderación o consultar los informes disponibles.	El sistema permite gestionar reportes, moderar contenido, suspender cuentas, administrar catálogos y consultar información de misiones, créditos y actividad, según los permisos establecidos.

22. Conclusión

BountyLine es un caso con la complejidad suficiente para aplicar lo visto en Bases de Datos 1 y 2: un modelo de 14 entidades con especialización y relaciones 1:1, 1:N y N:M, lógica de negocio

real dentro de MySQL para la retención y el pago de recompensas, control transaccional con manejo de excepciones y un modelo documental en MongoDB justificado por el tipo de contenido que maneja. La temática de videojuego hace que las reglas del negocio sean fáciles de explicar en la sustentación, y la división en fases con Scrum permite entregar por partes y corregir a tiempo.

Referencias

Bradshaw, S., Brazil, E. y Chodorow, K. (2019). MongoDB: The definitive guide (3.ª ed.). O'Reilly Media.

Chen, P. P. (1976). The entity-relationship model: Toward a unified view of data. ACM Transactions on Database Systems, 1(1), 9-36. https://doi.org/10.1145/320434.320440

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377-387. https://doi.org/10.1145/362384.362685

Elmasri, R. y Navathe, S. B. (2016). Fundamentals of database systems (7.ª ed.). Pearson.

Project Management Institute. (2021). Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK) (7.ª ed.). Project Management Institute.

Sadalage, P. J. y Fowler, M. (2012). NoSQL distilled: A brief guide to the emerging world of polyglot persistence. Addison-Wesley.

Schwaber, K. y Sutherland, J. (2020). La guía de Scrum. https://scrumguides.org

Silberschatz, A., Korth, H. F. y Sudarshan, S. (2019). Database system concepts (7.ª ed.). McGraw-Hill Education.